QATIP Intermediate AWS Lab 07 Managing AWS S3 Storage using Terraform

Lab Objectives....2
Teaching Points....2
Before you begin....2
Solution....3
Task 1 Create an S3 bucket....3
Task 2 Configure private and secure access....5
Task 3 Examine the files upload code....7
Task 4 Create an S3 lifecycle policy....9
Task 5 Generate a pre-signed S3 URL....11
Task 6 Use the pre-signed URL....12
Task 7 Lab Clean-Up....13

In this lab, you will create a private Amazon S3 bucket, upload local files with the correct content types, configure a lifecycle policy and share one object using a temporary pre-signed URL. Terraform runs locally in the VS Code PowerShell terminal. All resources use **us-west-2 Oregon**.

The lab follows the Azure guide's structure with **Try it yourself**, **Step-by-Step**, code explanations and verification. S3 stores objects directly in a bucket; there is no separate resource group, storage account or container to create. A pre-signed S3 GET URL for one object replaces the Azure container SAS exercise.

Lab Objectives

Use Terraform to create and manage:

- An S3 bucket with a globally unique name.
- Public access blocking, versioning, encryption and an HTTPS-only bucket policy.
- An S3 object for each local sample file, with the correct MIME type.
- A lifecycle configuration for transition and expiration.

Use the AWS CLI from PowerShell to generate a read-only URL for Teide.jpeg, then test it in a browser.

Teaching Points

This lab focuses on managing S3 storage and object metadata. You will examine fileset(), for_each, jsondecode(), filemd5(), regex(), try() and lookup(), then explore authenticated sharing.

Terraform manages the infrastructure and uploads. The AWS CLI signs the temporary URL locally using your existing credentials. That URL is not a Terraform-managed resource.

Before you begin

1. Complete Lab 0. Use the same lab account and authentication method for Terraform and the AWS CLI. You need VS Code, Terraform 1.10 or later and below 2.0, AWS CLI v2 and PowerShell 7.4 or later.
2. In VS Code choose **File > Open Folder**, open C:\aws-tf-int\labs\07, and open a PowerShell 7 terminal.
3. Run all commands from C:\aws-tf-int\labs\07. Use the AWS Console to verify results; change resources through Terraform. This lab uses local state. Keep state until cleanup is complete and do not commit it to Git.

Solution

A completed example is supplied at aws-tf-int\labs\solutions\07 in the test pack. Use it only as a last resort.

To use it after starting the lab, copy **only its main.tf** into your original working folder and retain your original bucket name. Keep the working state files there. Do not run the separate solution folder against an existing bucket: it has no state recording your deployment.

Task 1 Create an S3 bucket

Try it yourself

Populate Task 1 of main.tf with Terraform and AWS provider requirements, an AWS provider for us-west-2, an aws_s3_bucket resource called storage, and a bucket_name output. Use a unique bucket name containing your initials or student ID. Save, initialise, plan, apply and verify.

Step-by-Step

1. Open main.tf. Leave Task 3 commented out for now.
2. Add this beneath the **Task 1** heading:

```
terraform {
 required_version = ">= 1.10.0, < 2.0.0"
 required_providers {
  aws = {
   source  = "hashicorp/aws"
   version = "~> 6.0"
  }
 }
}
provider "aws" {
 region = "us-west-2"
}
resource "aws_s3_bucket" "storage" {
 bucket = "REPLACE_WITH_UNIQUE_BUCKET_NAME"
 # Disposable lab bucket only: remove all versions during terraform destroy.
 force_destroy = true
```

```
 tags = {
   Name   = "Terraform storage lab"
   Course = "QATIPINT"
   Lab    = "07"
 }
}
output "bucket_name" {
 value = aws_s3_bucket.storage.id
}
```

3. Replace REPLACE_WITH_UNIQUE_BUCKET_NAME with your own lowercase name, for example qatipint-mcg-static-482761. Use letters, digits and hyphens, between 3 and 63 characters. The example may already be taken.
4. force_destroy = true allows Terraform to remove objects and old versions when destroying this disposable lab bucket. Use only the supplied samples; this deliberately destructive setting makes lab cleanup possible.
5. Save and run. The supplied lock file selects AWS provider 6.72.0 within the version 6 constraint:

```
terraform fmt
terraform init
terraform validate
terraform plan
terraform apply
```

6. Review the plan. It should create **one bucket**. Enter yes when prompted by terraform apply.
7. Capture its name for later checks:

```
$bucketName = terraform output -raw bucket_name
$bucketName
```

8. In the AWS Console open **S3 > General purpose buckets**, locate the bucket and confirm **US West Oregon us-west-2**.

Checkpoint: the bucket exists and the output contains its exact name. Keep this name unchanged for the remaining tasks.

The provider's region replaces the original starter's us-east-1 setting. AWS uses your existing credentials; there is no Azure features {} block or subscription ID to add.

Task 2 Configure private and secure access

Try it yourself

In Task 2, add all four public access block settings, enabled versioning, default AES256 encryption using S3 managed keys, and a policy denying requests when aws:SecureTransport is false. Apply the policy to bucket and object ARNs. Save, plan, apply and verify.

Step-by-Step

1. Add this beneath the **Task 2** heading:

```
resource "aws_s3_bucket_public_access_block" "storage" {
 bucket = aws_s3_bucket.storage.id
 block_public_acls      = true
 block_public_policy    = true
 ignore_public_acls     = true
 restrict_public_buckets = true
}
resource "aws_s3_bucket_versioning" "storage" {
 bucket = aws_s3_bucket.storage.id
 versioning_configuration {
  status = "Enabled"
 }
}
resource "aws_s3_bucket_server_side_encryption_configuration" "storage" {
 bucket = aws_s3_bucket.storage.id
 rule {
   apply_server_side_encryption_by_default {
    sse_algorithm = "AES256"
   }
 }
}
resource "aws_s3_bucket_policy" "https_only" {
 bucket = aws_s3_bucket.storage.id
```

© QA 2026

```
 policy = jsonencode({
  Version = "2012-10-17"
  Statement = [{
   Sid      = "DenyInsecureTransport"
   Effect   = "Deny"
   Principal = "*"
   Action   = "s3:*"
   Resource = [
    aws_s3_bucket.storage.arn,
    "${aws_s3_bucket.storage.arn}/*"
   ]
   Condition = {
    Bool = {
     "aws:SecureTransport" = "false"
    }
   }
  }]
 })
}
```

2. Each configuration identifies the bucket using aws_s3_bucket.storage.id. The policy covers its bucket ARN and the object ARN ending in /*. jsonencode() converts the Terraform expression into JSON.
3. The explicit deny applies when transport is not secure. It does not grant anonymous access; public access blocking stays enabled.
4. Save and run:

```
terraform fmt
terraform validate
terraform plan
terraform apply
```

5. Review the four configuration resources being added and enter yes.
6. Open your bucket in the AWS Console:
   - Under **Properties**, verify **Bucket Versioning** is enabled and default encryption is **SSE-S3**.

© QA 2026

- Under **Permissions**, verify all public access block settings are on and the policy contains DenyInsecureTransport.

7. Check versioning in PowerShell:

   ```
   aws s3api get-bucket-versioning --bucket $bucketName --region us-west-2 --no-cli-pager
   ```

   Expected status: Enabled.

Checkpoint: the bucket is private, versioned, encrypted and has an explicit deny for unencrypted transport.

After first enabling versioning, AWS recommends waiting **15 minutes** before uploading objects because the change can take time to propagate. Use this interval to examine Task 3 before applying the uploads. See the versioning reference at the end.

Task 3 Examine the files upload code

Try it yourself

Uncomment the pre-populated Task 3 code. Examine how it uploads the three files and assigns their MIME types. Save, plan, apply and verify each content type.

Step-by-Step

1. Open mime.json. Relevant entries include:

   ```
   {
     ".txt": "text/plain",
     ".jpeg": "image/jpeg",
     ".png": "image/png"
   }
   ```

   Keep the complete supplied file; this excerpt just shows the mappings used by the samples.

2. Remove the leading # from the Task 3 code lines in main.tf. Leave the task heading as a comment. The result should be:

```
locals {
 mime_types = jsondecode(file("${path.module}/mime.json"))
}
resource "aws_s3_object" "upload_files" {
 for_each = fileset("${path.module}/static_files", "**/*")
 bucket     = aws_s3_bucket.storage.id
 key        = each.key
 source     = "${path.module}/static_files/${each.key}"
 source_hash = filemd5("${path.module}/static_files/${each.key}")
 content_type = lookup(
  local.mime_types,
  lower(try(regex("\\.[^.]+$", each.key), "")),
  "application/octet-stream"
 )
 depends_on = [
  aws_s3_bucket_versioning.storage,
  aws_s3_bucket_server_side_encryption_configuration.storage
 ]
}
```

Apply and verify

3. Once first-time propagation interval described in Task 2 has passed, run:

```
terraform fmt
terraform validate
terraform plan
terraform apply
```

4. Review the **three objects** being created and enter yes.
5. In the AWS Console open the bucket's **Objects** tab, select each object and inspect **Properties** and **Metadata**:
6. Check one object in PowerShell:

```
aws s3api head-object --bucket $bucketName --key Teide.jpeg --region us-west-2 --query '{ContentType:ContentType,Encryption:ServerSideEncryption,VersionId:VersionId}' --output json --no-cli-pager
```

Expect image/jpeg, AES256 and a version ID. A missing StorageClass field for a new object generally means S3 Standard.

Checkpoint: all three objects exist with correct content types. Keep the bucket private; sharing is covered in Task 5.

Task 4 Create an S3 lifecycle policy

Try it yourself

Create one aws_s3_bucket_lifecycle_configuration with one enabled rule, archive-and-expire:

- Use an empty filter to cover all keys.
- Transition eligible current objects to GLACIER after 30 days.
- Expire current objects after 90 days.
- Permanently remove noncurrent versions 90 days after they become noncurrent.

Save, plan, apply and verify the configuration. You do not wait for objects to age during the lab.

Step-by-Step

2. Add this beneath the **Task 4** heading:

```
resource "aws_s3_bucket_lifecycle_configuration" "storage" {
 bucket = aws_s3_bucket.storage.id
 rule {
  id     = "archive-and-expire"
  status = "Enabled"
  filter {}
  transition {
   days          = 30
   storage_class = "GLACIER"
  }
  expiration {
   days = 90
  }
  noncurrent_version_expiration {
   noncurrent_days = 90
  }
 }
```

```
  depends_on = [aws_s3_bucket_versioning.storage]
}
```

2. Save and run:

```
terraform fmt
terraform validate
terraform plan
terraform apply
```

3. Review the single lifecycle configuration being added and enter yes.
4. In the AWS Console open **Management > Lifecycle rules** and inspect archive-and-expire.
5. Retrieve the configuration:

```
aws s3api get-bucket-lifecycle-configuration --bucket $bucketName --region us-west-2 --no-cli-pager
```

Versioning and expiration: at day 90, current-object expiration adds a delete marker. It does not immediately erase the data version. The noncurrent action permanently removes a version after it has been noncurrent for 90 days. This does not permanently delete every version exactly 90 days after its original upload.

Small objects: S3's default transition behavior excludes objects smaller than 128 KB. The text file can expire but will not transition under that default. Both supplied images exceed the threshold. Keep the default for this exercise.

Lifecycle actions are asynchronous. The browser test uses objects still in S3 Standard. An object in Glacier Flexible Retrieval needs restoration before normal download, and that class has a 90-day minimum storage duration charge. Complete cleanup during this session rather than retaining the bucket to observe the transition.

Checkpoint: the rule exists with the expected actions, and you understand current expiration versus noncurrent version deletion.

Task 5 Generate a pre-signed S3 URL

Try it yourself

Add outputs for the S3 URI and unsigned HTTPS URL of Teide.jpeg. Apply them, then generate a read-only pre-signed URL with a requested lifetime of one hour.

Step-by-Step

1. Add this beneath the **Task 5** heading:

```
output "object_uri" {
 value =
"s3://${aws_s3_bucket.storage.id}/${aws_s3_object.upload_files["Teide.jpeg"].key}
"
}
output "object_https_url" {
 value = "https://${aws_s3_bucket.storage.id}.s3.us-west-
2.amazonaws.com/Teide.jpeg"
}
```

2. Save and run:

```
terraform fmt
terraform validate
terraform plan
terraform apply --auto-approve
```

   The plan should add outputs without changing infrastructure.

3. Capture the S3 URI:

```
$objectUri = terraform output -raw object_uri
$objectUri
```

   Expected form: s3://your-bucket-name/Teide.jpeg.

4. Generate the URL:

```
$signedUrl = aws s3 presign $objectUri --expires-in 3600 --region us-west-2
```

   Check that the command succeeded. The signing identity needs s3:GetObject permission for this object.

5. Display the URL when ready:

©QA 2026

$signedUrl

The URL authorizes GET for this one object. It does not grant list or write access. Its requested lifetime is 3,600 seconds; temporary signing credentials may expire sooner and end access early.

No SAS data source is added to Terraform. There is no sensitive Terraform output for this URL: Terraform outputs only object addresses, and the complete signed URL is held in PowerShell. Treat it as a temporary credential. Do not commit it to Git or include it in code screenshots.

Checkpoint: $signedUrl contains an HTTPS address with signature and expiry parameters. It is generated on demand, not stored as a resource in S3 or Terraform state.

Task 6 Use the pre-signed URL

Step-by-Step

1. Obtain the unsigned address:

   ```
   $unsignedUrl = terraform output -raw object_https_url
   $unsignedUrl
   ```

2. Paste it into a private browser window. Anonymous access should return **AccessDenied**. An AWS Console login does not authenticate an ordinary unsigned object URL.

3. Open the signed URL:

   ```
   Start-Process $signedUrl
   ```

   The browser should display Teide.jpeg. You can also paste the complete signed URL into the private window. Keep its entire query string and use HTTPS.

4. If the instructor requests it, share the signed link to this sample image in the class chat for verification. Anyone with the link can use it while valid.

5. To test expiry during the session, create a separate short-lived link:

```
$shortUrl = aws s3 presign $objectUri --expires-in 60 --region us-west-2
$shortUrl
```

Open it immediately. After more than 60 seconds, use this new GET request to avoid relying on a cached image:

```
Invoke-WebRequest -Uri $shortUrl -Method Get
```

Expect an S3 expiry error. Use GET, not HEAD: this link was signed for GET. Do not change the Windows clock. An existing download may finish after expiry; expiry is checked when a new request begins.

Checkpoint: unsigned access fails, signed access retrieves the image, and a new request using an expired link fails.

Task 7 Lab Clean-Up

1. Return to the folder containing the state used to create your resources:

   ```
   Set-Location C:\aws-tf-int\labs\07
   terraform plan -destroy
   ```

2. Confirm the plan targets only the lab bucket, its configuration and objects. Run:

   ```
   terraform destroy
   ```

   Review and enter yes.

   The bucket's force_destroy = true setting permits deletion of remaining versions and delete markers. Deleting only visible current objects would not empty a versioned bucket.

3. Verify the bucket is gone in the AWS Console. Keep code for review, but do not share state or signed URLs.

4. Remove temporary link variables:

   ```
   Remove-Variable signedUrl, shortUrl -ErrorAction SilentlyContinue
   ```

Congratulations, you have completed this lab.

©QA 2026

Troubleshooting

Symptom	Check
Bucket name already in use	Choose another globally unique lowercase name before the first successful apply.
AccessDenied during apply	Check the active identity and the specific denied S3 action with the instructor.
Wrong content type	Check the extension, MIME map and applied Task 3 code.
Regex errors	Include try() around regex() and the lookup() fallback.
Missing object after first enabling versioning	Allow the Task 2 interval, check the key and retry.
Signed URL returns AccessDenied or ExpiredToken	Check read permission and current session credentials; refresh and generate a new link.
SignatureDoesNotMatch	Use the complete HTTPS URL and GET method. Check clock accuracy and proxy changes.
No immediate Glacier transition	Actions are age-based and asynchronous; the default excludes objects below 128 KB. Verify the rule, then clean up.
BucketNotEmpty on destroy	Confirm force_destroy = true was successfully applied and you can list and delete versions. Retain state and retry after resolving the cause.

Reference links

AWS provider
S3 bucket
S3 public access block
S3 versioning
S3 encryption
S3 bucket policy
S3 object and source_hash
S3 lifecycle configuration
AWS versioning propagation guidance
S3 lifecycle actions and versions
S3 lifecycle transition considerations
AWS CLI presign
Pre-signed URL permissions and expiration

© QA 2026

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026